Assignment 3: Software-Defined Networking (SDN)

COL7524: Advanced Networks

Provided Reference Files

The following topology files are provided as starting points; controller applications are not provided.

- `p1_topo.py`: a two-switch topology with five hosts, intended for Part 1. Hosts h1–h3 connect to s1 and h4–h5 connect to s2; the switches are joined by a bandwidth-limited link.
- `p3_topo.py`: a four-switch ring topology with four hosts, intended for Parts 2 and 3. It provides two alternative routes between opposite switches. The inter-switch links have bandwidths of 2, 2, 1, and 1 Mbps and delays of 20, 10, 20, and 15 ms, respectively.

You may modify a provided topology or create your own if needed. Include all topology files used in your experiments in your submission and briefly state any changes made.

Required Environment

Use Linux (Ubuntu is recommended) with Python 3, Mininet, Open vSwitch with OpenFlow 1.3 support, and a Ryu version compatible with the selected Python/OpenFlow versions. Install `iperf` if using throughput as a performance measure. The topologies connect to a controller at `127.0.0.1:6633` by default. Start Ryu and Mininet in separate terminals; Mininet typically requires `sudo`. Keep the controller address and port consistent. No specific versions are mandated, but the selected versions must work together.

Part 1: Reactive and Proactive Flow Management

Compare two approaches to installing forwarding rules in an SDN network:

- **Reactive installation:** the controller installs a forwarding rule when traffic for a flow first arrives and requires a forwarding decision.
- **Proactive installation:** the controller installs the required forwarding rules before the corresponding traffic begins.

Implement both approaches using `p1_topo.py` or a documented modification. Ensure that hosts can communicate and inspect the switch flow tables. Compare the approaches using one suitable measure, such as throughput or the number of Packet-In messages handled by the controller. Explain the main trade-off between initial forwarding delay, controller workload, and rule installation.

Part 2: Link-Metric-Based Path Selection

Develop an SDN controller that selects routes between hosts based on link properties. Use `p3_topo.py` or another documented topology with at least two possible paths between a source and destination.

The controller should represent the switch-level topology, identify host attachment points, calculate link costs using **one metric of your choice** (bandwidth or delay), compute a path, and install the required OpenFlow rules along it. A higher-bandwidth link should have a lower cost when bandwidth is the chosen metric; when using delay, lower-delay links should have a lower cost. Ensure that basic connectivity works in both directions.

Explain why a particular path is selected. Change a link cost or link property in one test and check whether the routing decision changes as expected. Support your explanation with selected-path or flow-table evidence and one suitable performance measure, such as delay or throughput.

Part 3: Congestion-Aware Routing

Extend the Part 2 controller so that route selection can account for changing link utilization, rather than relying only on static link properties. The objective is to respond when a previously suitable path becomes congested.

Use switch statistics or another clearly described mechanism to estimate link utilization. Implement a simple congestion rule (for example, changing the cost of a link when its utilization exceeds a chosen threshold) and use it when selecting paths for **new flows**. You are not required to reroute existing flows. Your rule installation should allow a later flow to be assigned using the updated measurements.

Create one traffic scenario that congests a link and compare the selected path and one performance measure against static routing. Briefly explain the controller's decision and note one limitation of your approach.

Deadline and Submission Policy

- **Submission deadline: 12 Nov 2026.**

NOTE: Please make sure to press the submit button after uploading. No requests in this regard will be entertained.

Submission Instructions

Submit a single ZIP archive containing:

- **Source code:** Controller applications and all required topology, configuration, and experiment files.
- **Report (PDF):** Briefly describe the design and assumptions for each part. Include relevant flow-table or selected-path evidence, a small number of performance results, and explanations of the behavior observed.

The submission must be self-contained and runnable after downloading and extracting it in the required Linux environment. A separate README is not required; include essential setup or run commands briefly in the report or as comments in the relevant scripts. Do not assume that files outside the submitted archive will be available during evaluation.

Name the archive:

`<entry no 1> <entry no 2> A3.zip`

Requirements and Important Notes

- To keep the scope manageable, Part 2 requires only one static link metric, and Part 3 only needs to adapt routing for new flows in one congestion scenario.
- Students may work individually or in groups of **at most two members**.
- Students should be able to run each controller and explain its decisions using the observed network behavior and installed rules.
- Keep the analysis focused on the key comparison in each part; extensive collections of graphs are not required.
- Topology changes are allowed, but must be documented. State assumptions clearly and ensure experiments are reproducible.
- All submissions will be checked for plagiarism.